Variational Autoencoders

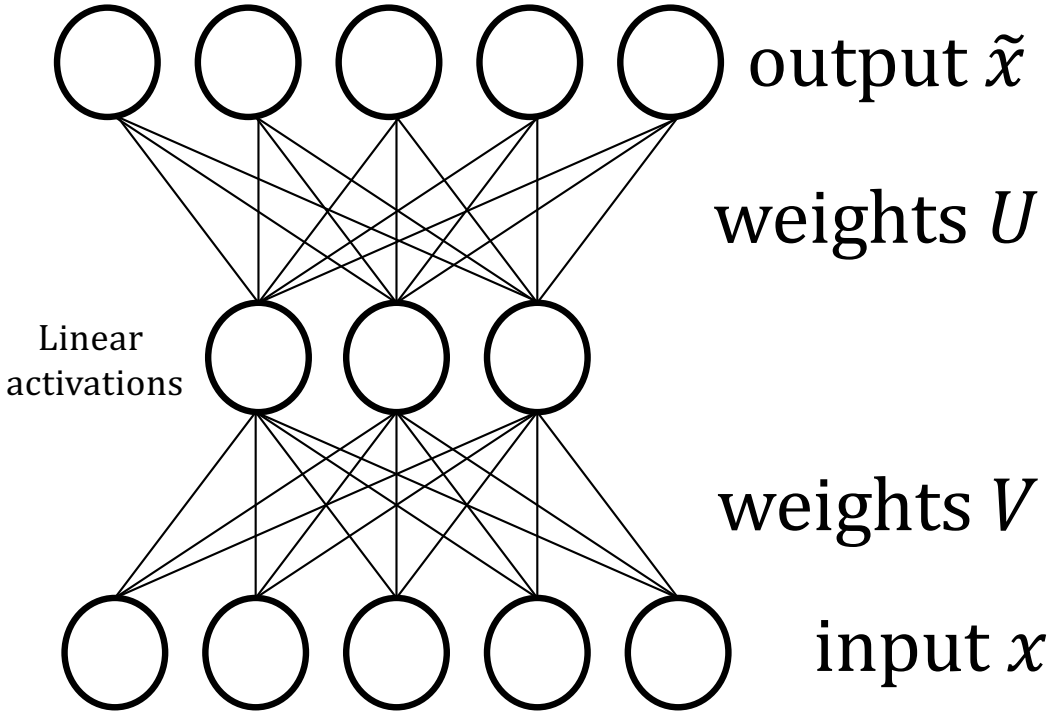
Bryan Pardo

Northwestern University

(updated fall 2026)

Reminder about Autoencoders

Simplest Autoencoder: Linear model, 1 hidden layer



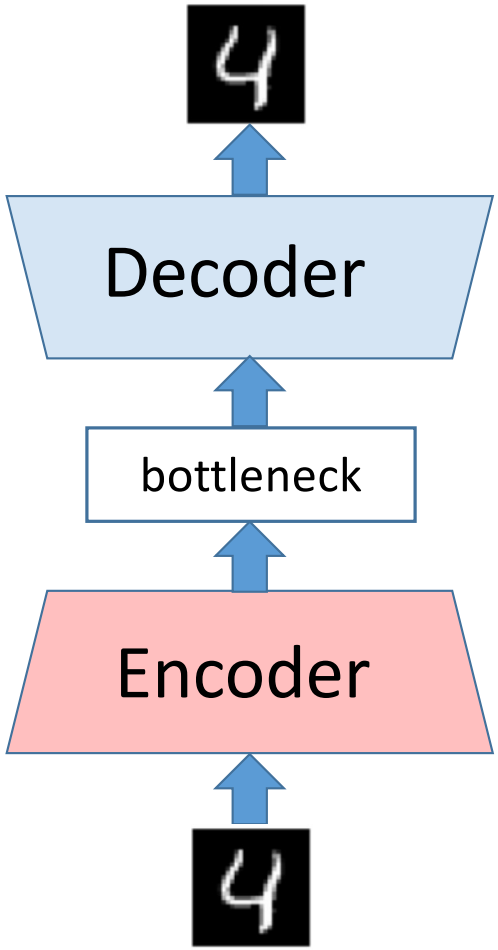
$$\text{loss } \mathcal{L} = \|x - \tilde{x}\|^2$$

$$\tilde{x} = UVx$$

Maps d dimensional input x to k dimensional embedding subspace S

More generally

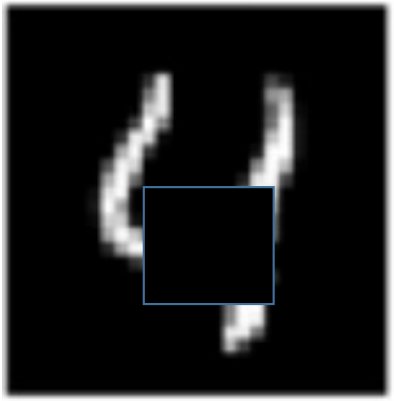
- With multiple layers & nonlinear activations we can map on to a nonlinear embedding space
- We can represent complex data this way and use the encoder as the input to a supervised network
- This lets us learn features from unlabeled data, which is far easier to get than labeled data.



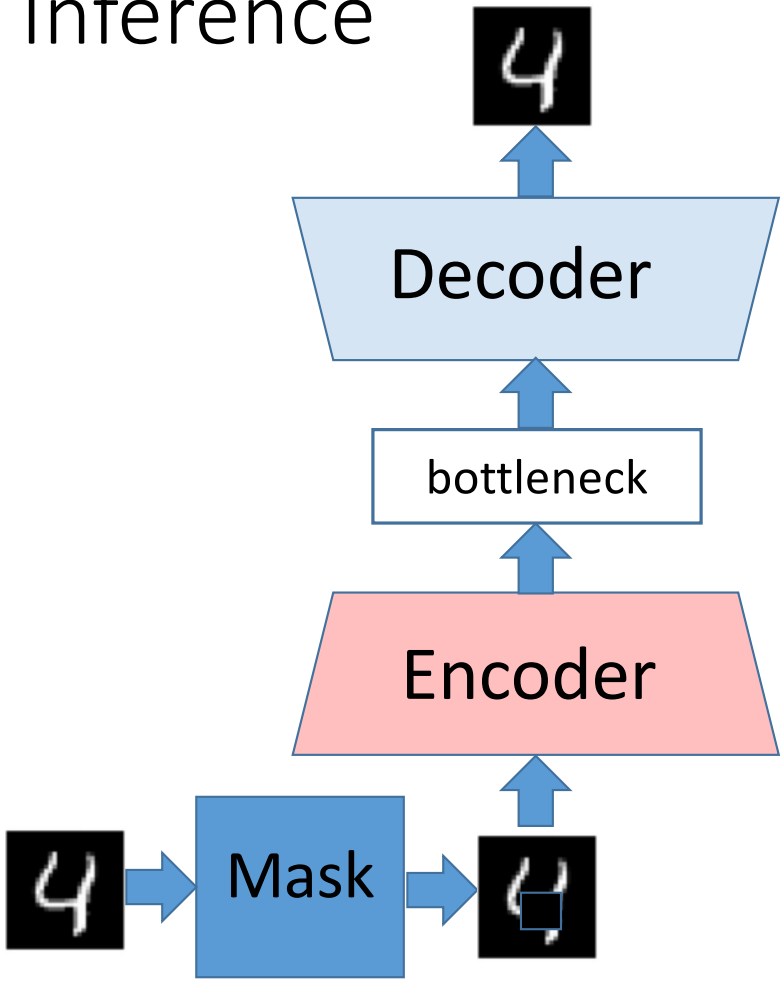
Using autoencoders for generation

Imputation/infill = Masked Inference

- If the “noise” we add is masking out large patches….

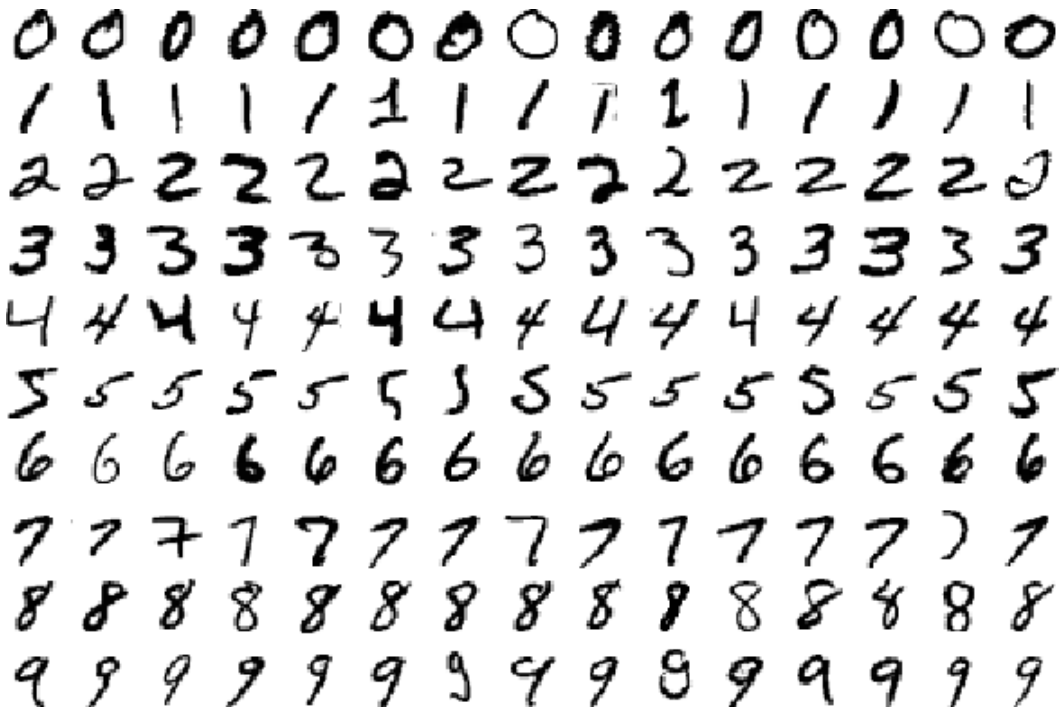


- We can train it to fill in blanks.



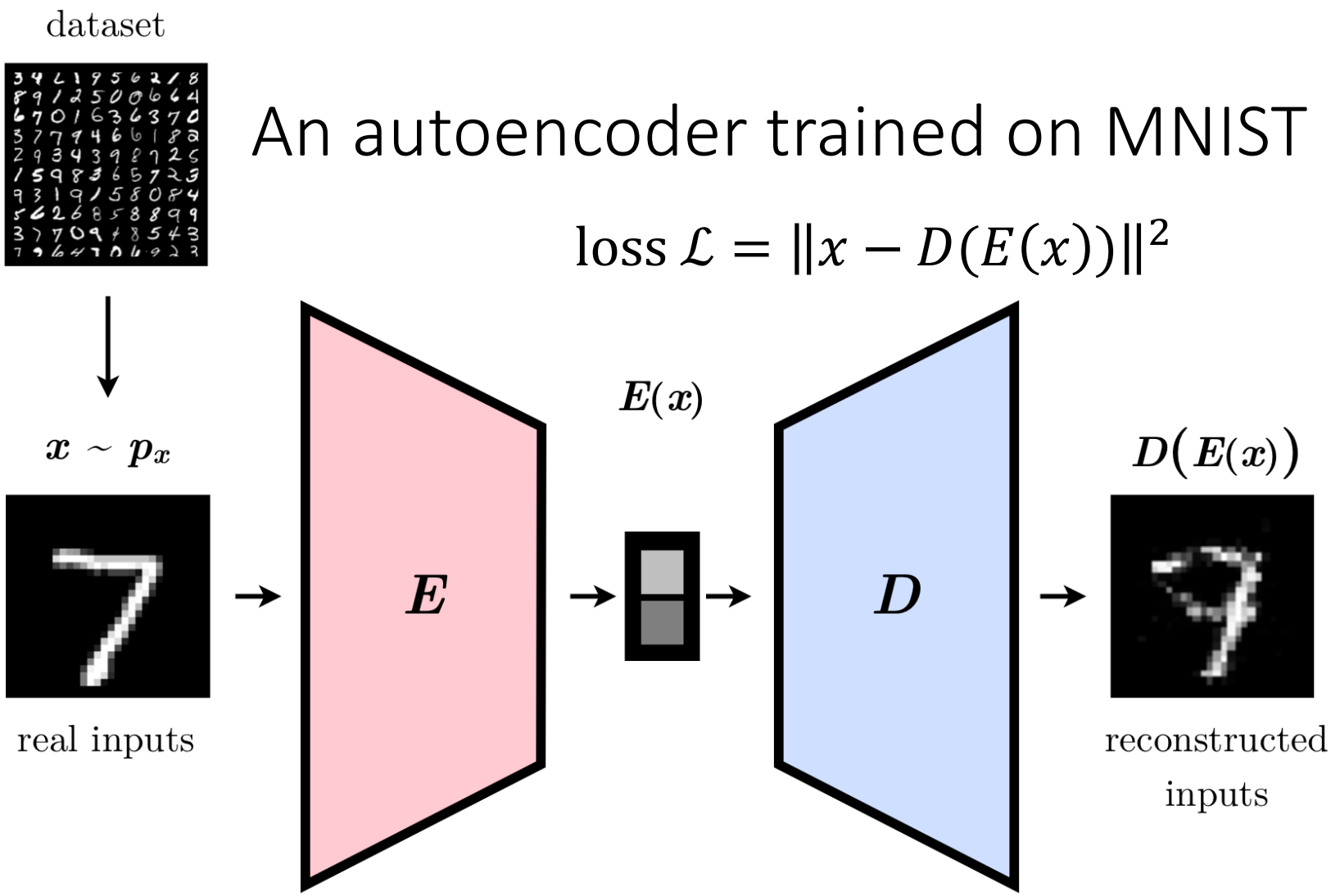
The MNIST dataset

- Famous dataset of handwritten digits
- They trained an autoencoder with a 2-dimensional bottleneck
- This makes visualizing the embedding space easy

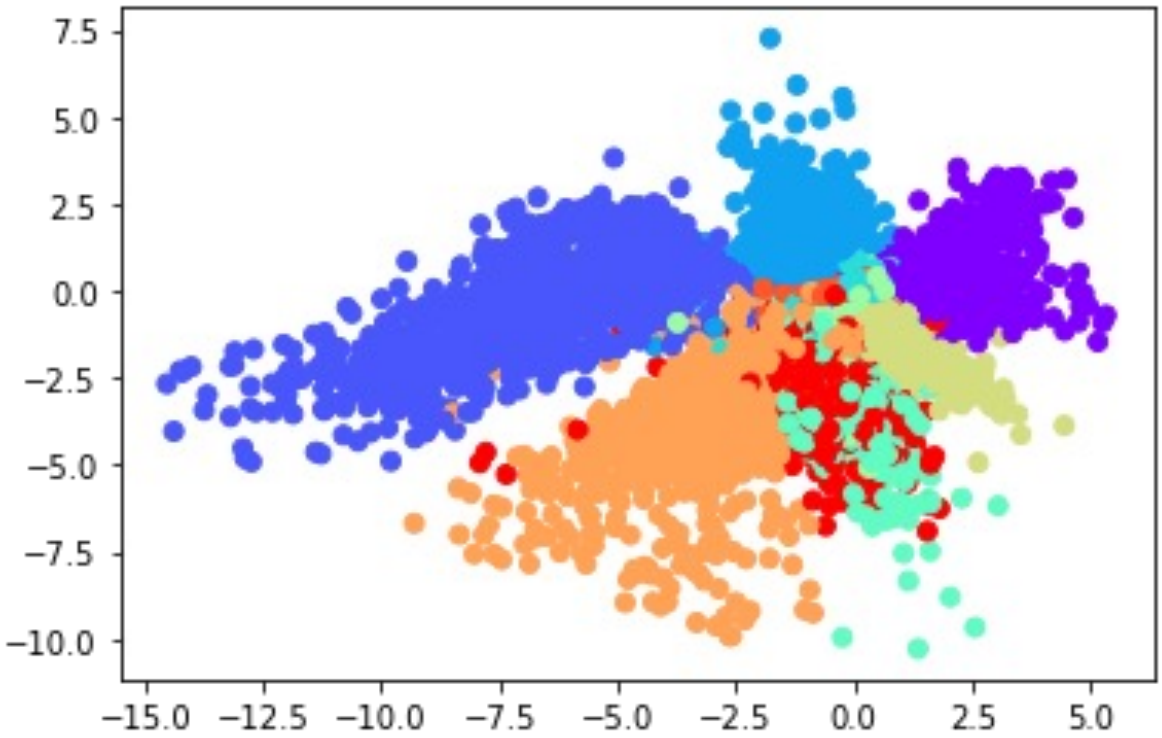


https://emkademy.medium.com/1-first-step-to-generative-deep-learning-with-autoencoders-22bd41e56d18

An autoencoder trained on MNIST



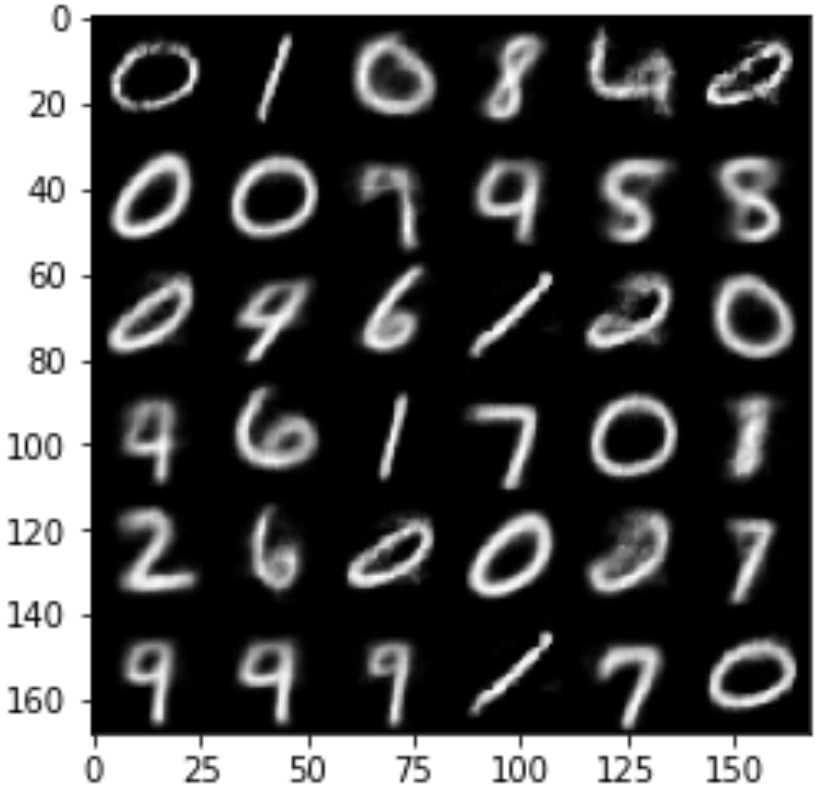
A 2-D “latent” space of the trained autoencoder



https://emkademy.medium.com/1-first-step-to-generative-deep-learning-with-autoencoders-22bd41e56d18

Generating images from “latent” vectors

- It is difficult to know what will happen when you move in a direction
- There are spots in the space that are…well…weird.



https://emkademy.medium.com/1-first-step-to-generative-deep-learning-with-autoencoders-22bd41e56d18

Variational Autoencoders

Based on (and figures from)

Kingma, Diederik P., and Max Welling. "An introduction to variational autoencoders."
Foundations and Trends® in Machine Learning 12.4 (2019): 307-392.
https://arxiv.org/pdf/1906.02691.pdf

…and

Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)
https://arxiv.org/pdf/1606.05908.pdf

…and

Odaibo, Stephen, “ Tutorial: Deriving the Standard Variational Autoencoder (VAE) Loss Function”
https://arxiv.org/pdf/1907.08956.pdf

A Bayesian Autoencoder

- Training goal: recreate the input dataset
- True objective: make things like the training data, but not in the training set
- **Hope: A smooth, continuous, interpretable latent space**

Some terminology

- If you can directly sample a random variable X and see the outcome x, we call x an **observation**. (e.g. the data our model is trained on)

- If you can't directly sample a random variable Z and see the outcome z, we call Z a **latent variable**.

- We'll typically use those letters with those implications: X is observable, Z is latent.

A simple latent variable model example

- What is the chance I have Covid, if I have a positive Covid test?
- Let Z be the latent variable ”Covid: yes/no”
- Let X be the observable variable “test: positive/negative”

A simple latent variable model example

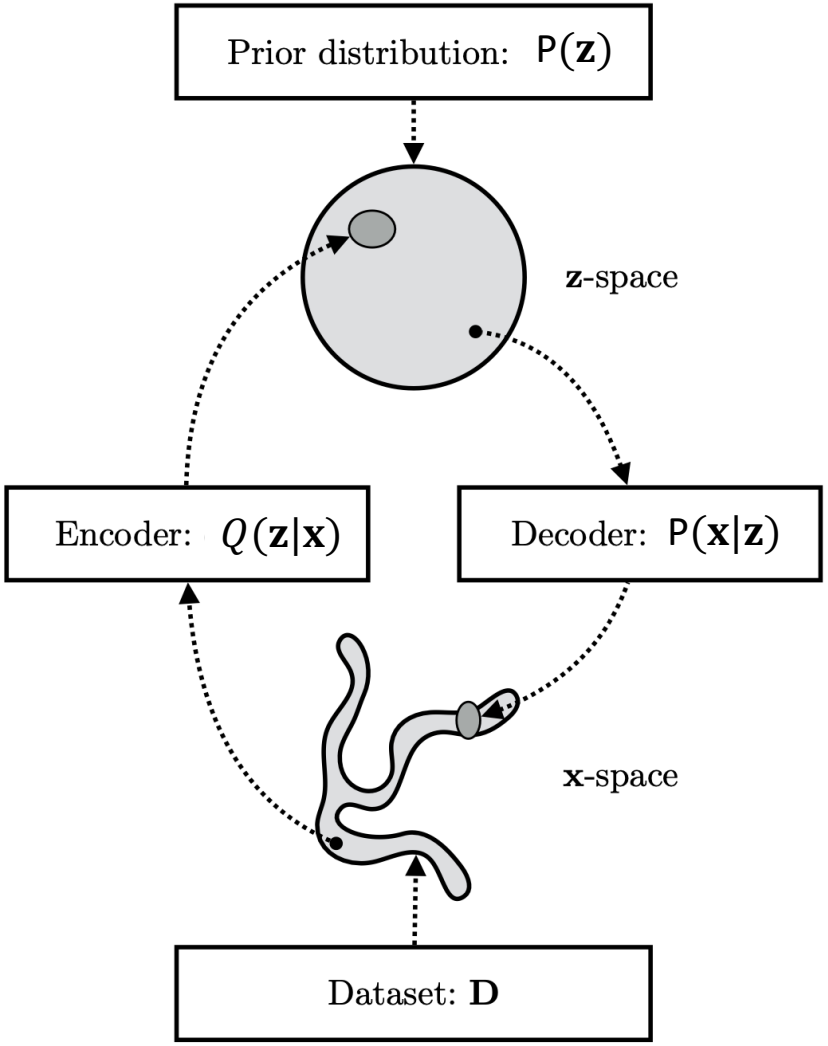
- I want to make a MNIST handwritten digit
- Let Z be the latent variable, you pick the dimensionality
- Let X be the observable variable: a handwritten digit. In this case, a 28 by 28 pixel image

The MNIST example

- P(X) is the unconditioned (aka **prior**) probability of the handwritten digits.
- P(Z) is the **prior** probability distribution of the latent control variable (this is something we just make up)
- P(Z|X) is the **posterior** probability of latent variable Z, given input image X
- P(X|Z) is the probability of outputting image X, given latent variable Z

A Variational Auto Encoder

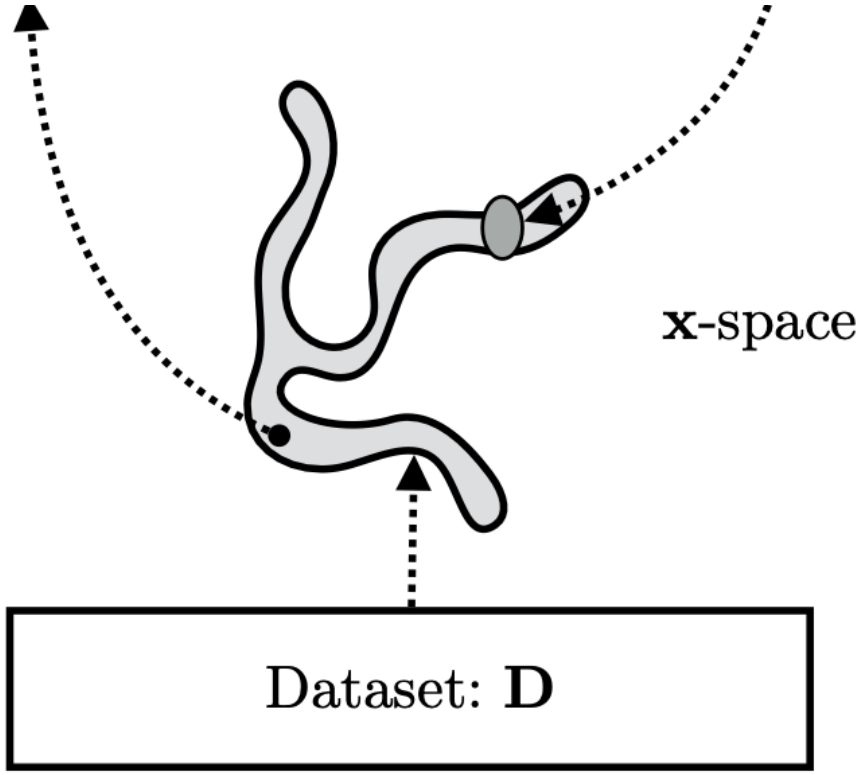
- An encoder/decoder framework
- We train the encoder to encode dataset samples as points in the latent space
- We train the decoder to decode points in the latent space into examples in the distribution of the training data



Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

The Evidence distribution P(**x**)

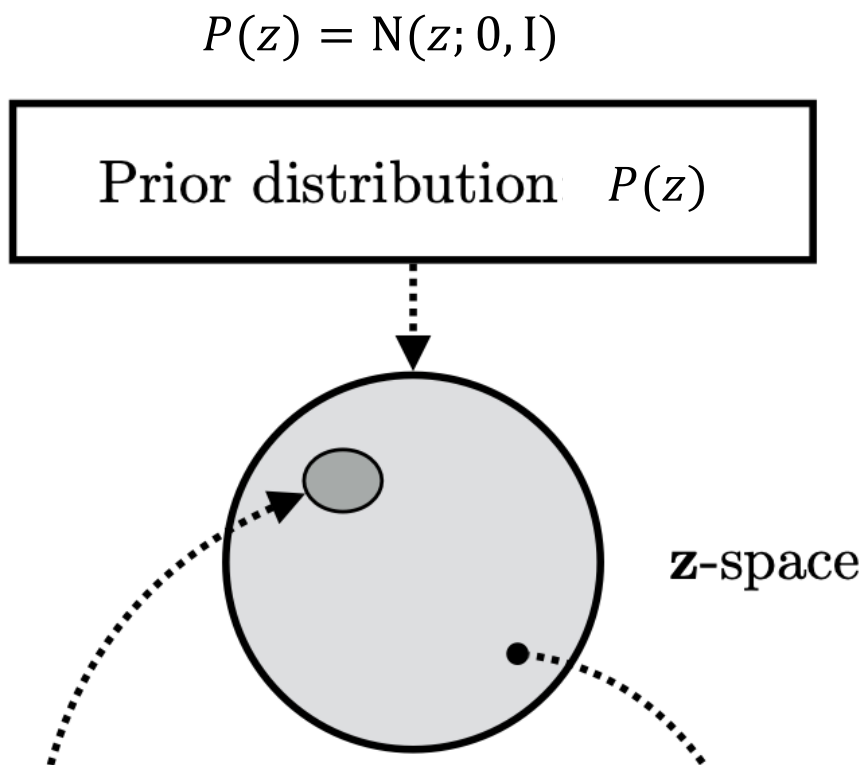
- The Dataset (aka the Evidence) contains examples **x** drawn from the **unknown** distribution P(**x**)
- Examples could be pictures, sounds, whatever.
- If we already had a good estimate of $\mathrm{P}(\mathbf{x})$ we wouldn't have to build a VAE. We'd just sample from it.



Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

The ”true” prior latent distribution $P(z)$

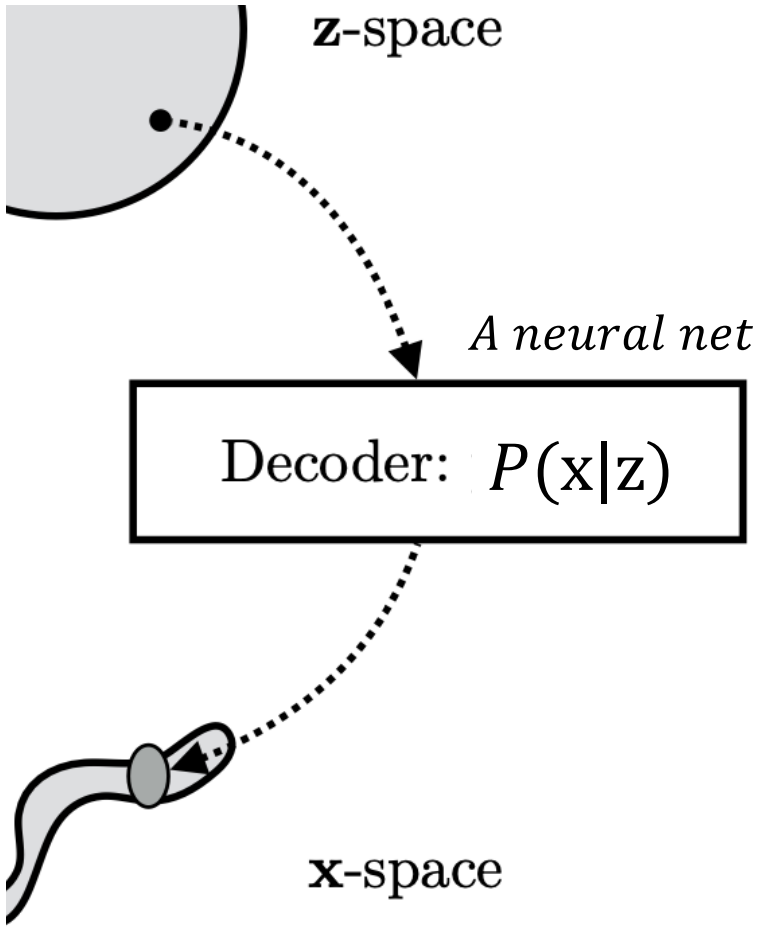
- We get to pick what this is.
- To make life easier, we’ll make $P(z)$ a Normal (Gaussian) distribution.
- Let’s specify $\mu = 0, \Sigma = \mathbf{I}$,the identity matrix. (a spherical, 0-centered distribution)



Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

The Decoder

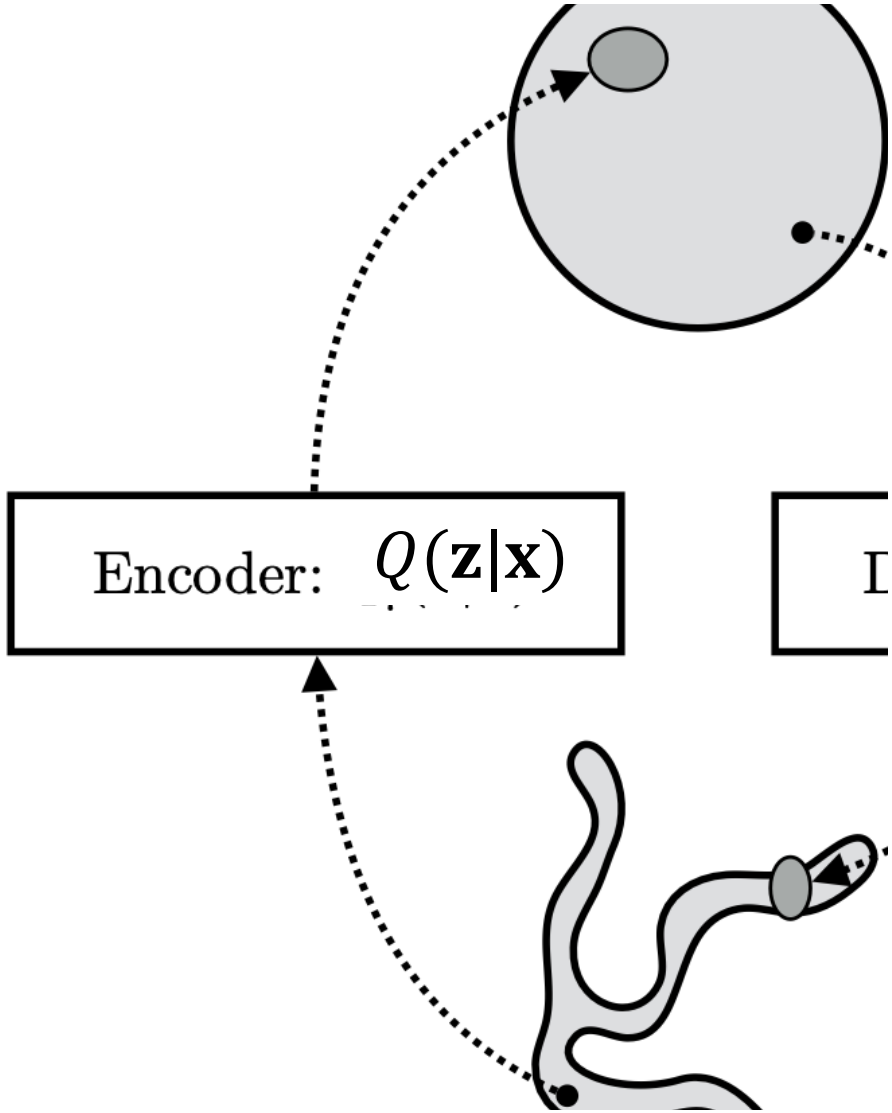
- The Decoder is a neural network with parameters we'll be learning.
- It maps a latent sample z into an example x.
- It outputs a probability distribution for x, then we argmax sample from that distribution to generate a sample.
- The Decoder is used to generate new examples (i.e. handwritten digits).



Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

The Encoder $Q(\mathbf{z}|\mathbf{x})$

- The Encoder is a neural network we'll be learning the parameters for.
- It approximates $P(\mathbf{z}|\mathbf{x})$, the true conditional distribution of the latents, given the data.
- In our running example X is a handwritten digit.



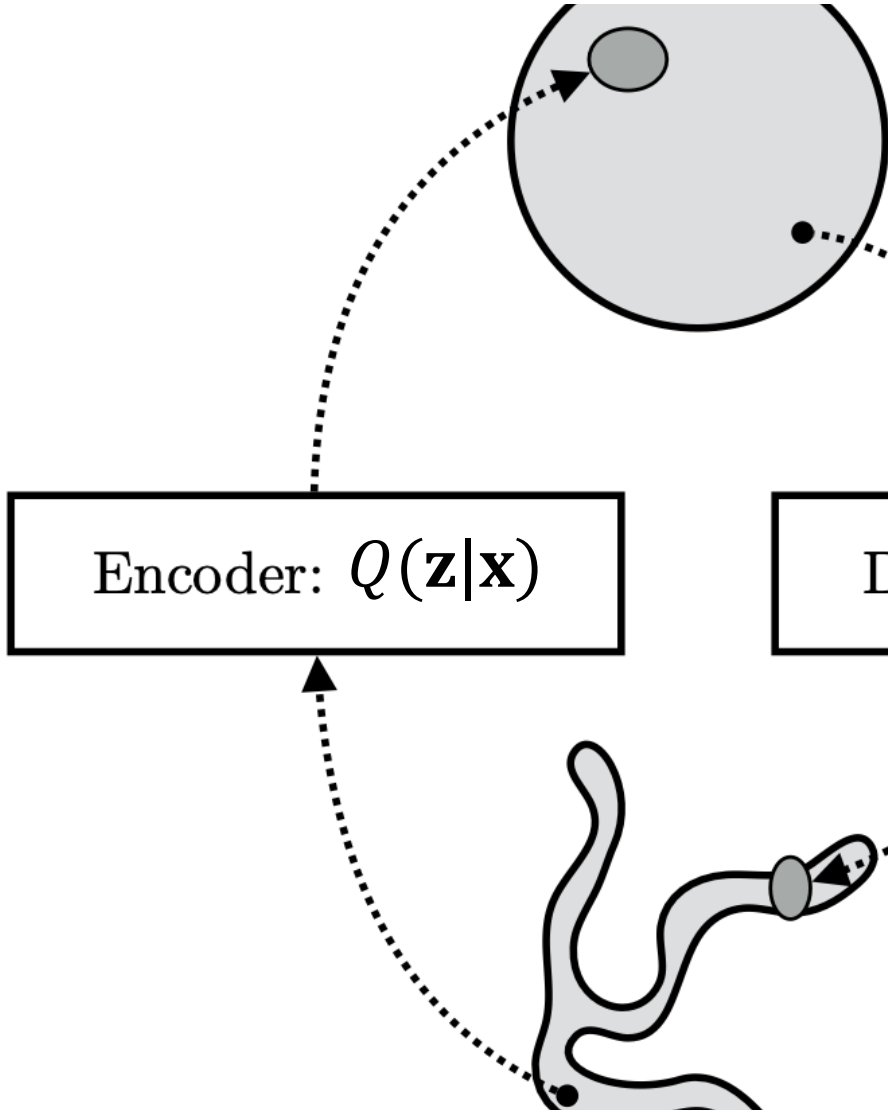
Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

The Encoder $Q(\mathbf{z}|\mathbf{x})$

- The Encoder embodies a normal distribution:

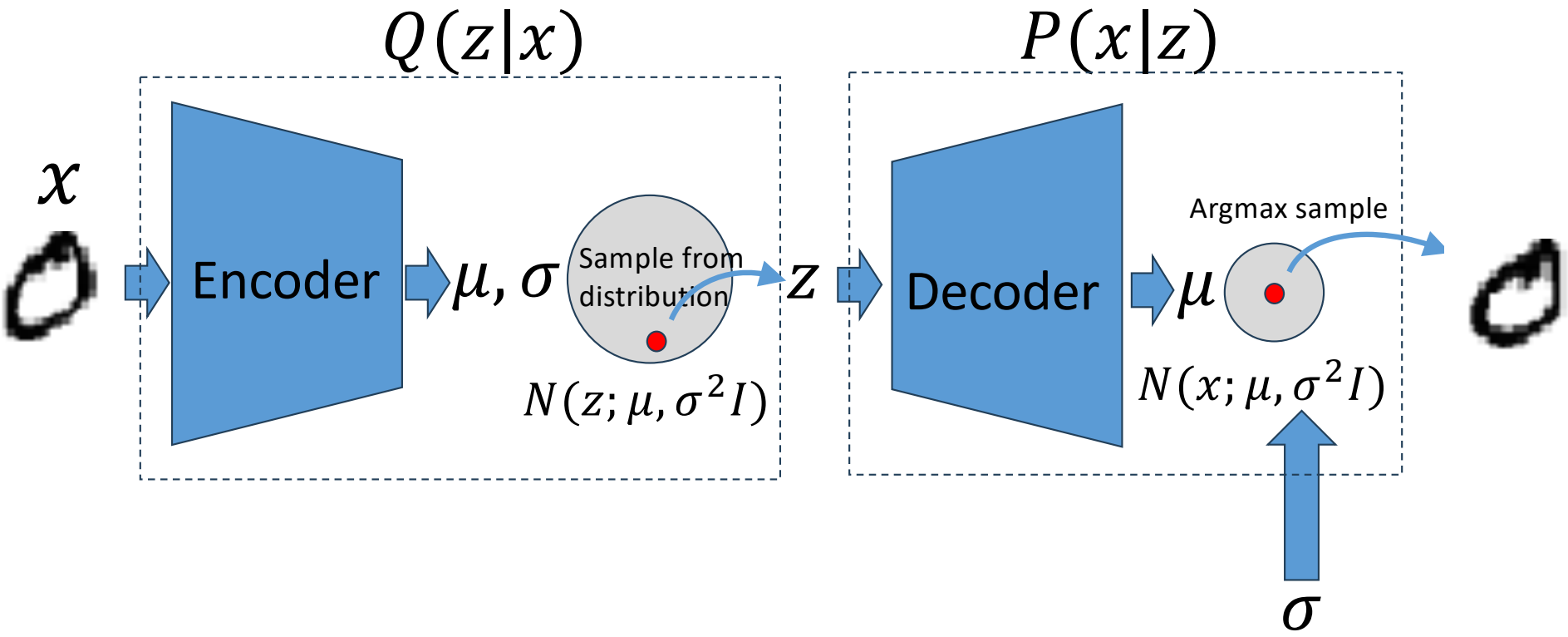
$$Q(z|x) = N(z; \mu, \sigma^2 I)$$

- The network outputs: a mean vector μ and the standard deviation σ^2.
- We sample a latent vector from this distribution.

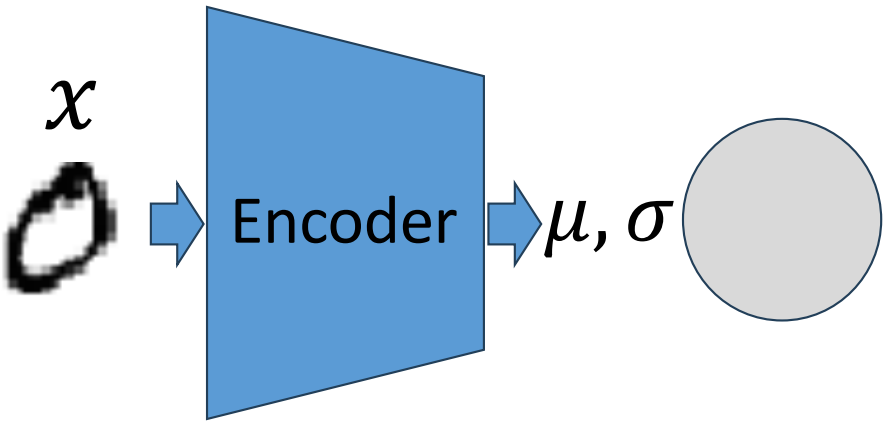


Adapted from Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

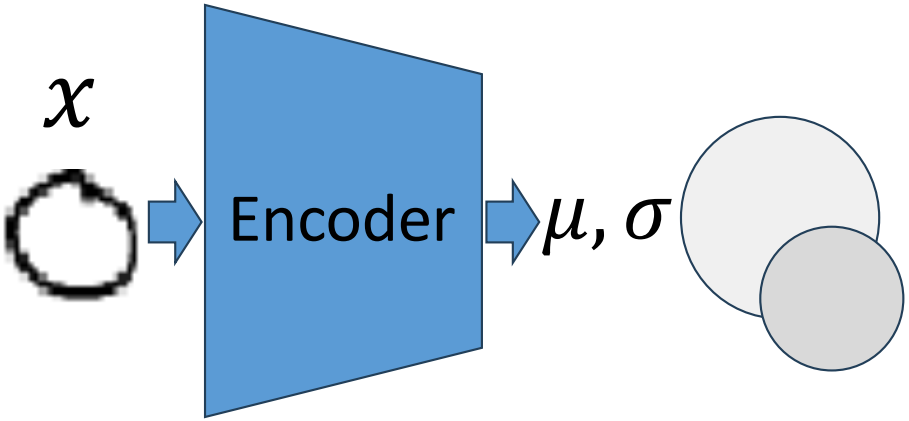
Running a single sample through the process



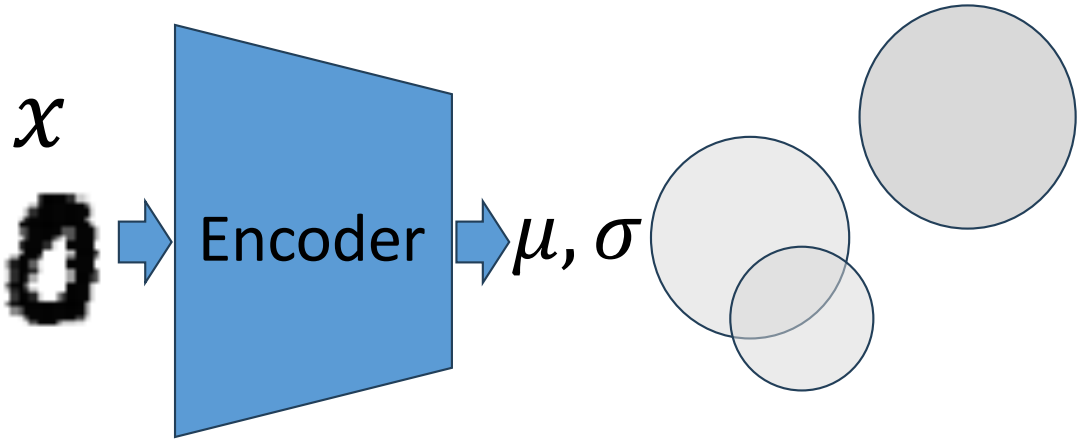
Different inputs result in different Gaussians



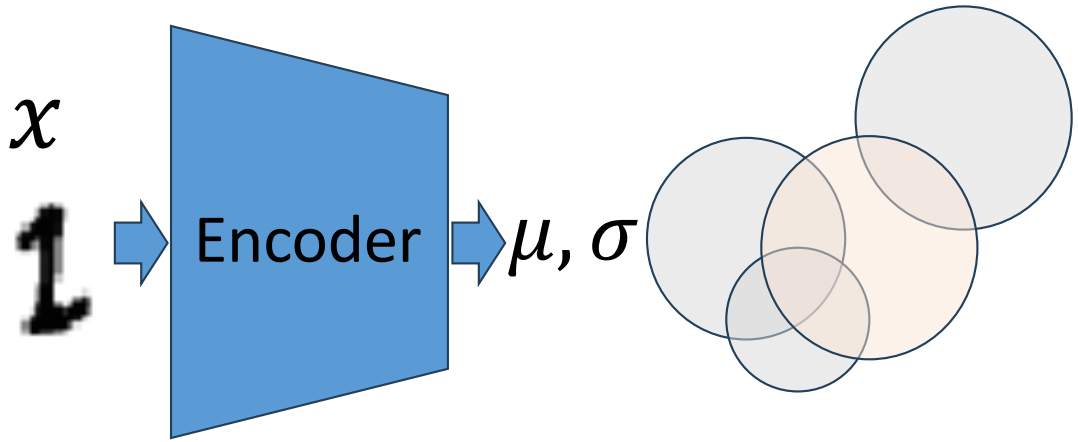
Different inputs result in different Gaussians



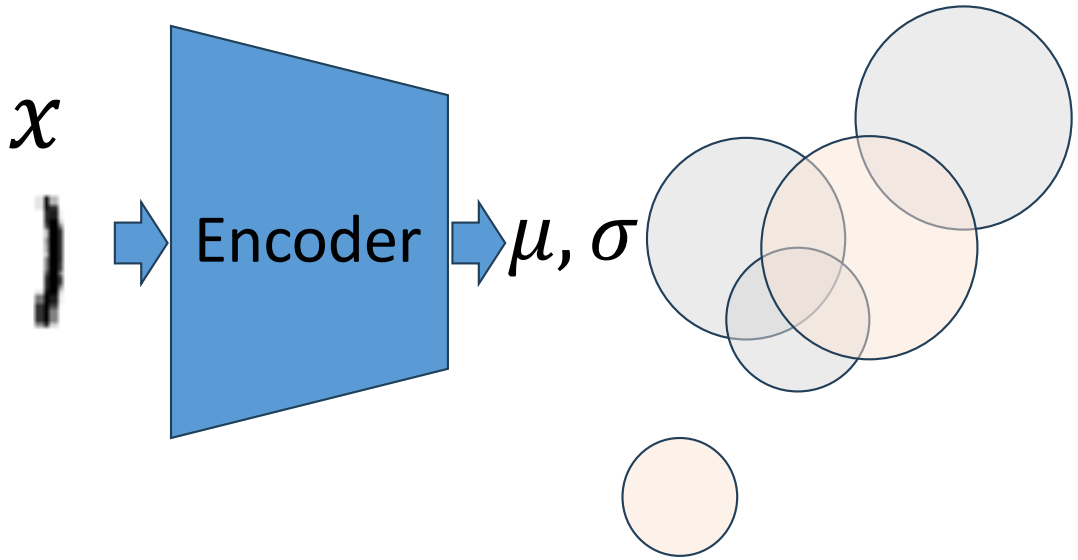
Different inputs result in different Gaussians



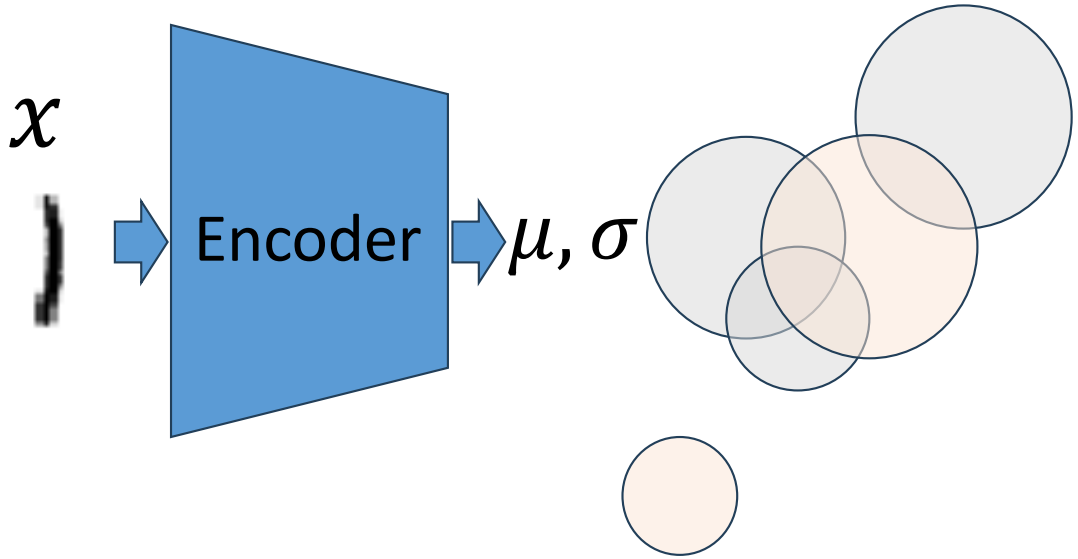
Different inputs result in different Gaussians



Different inputs result in different Gaussians

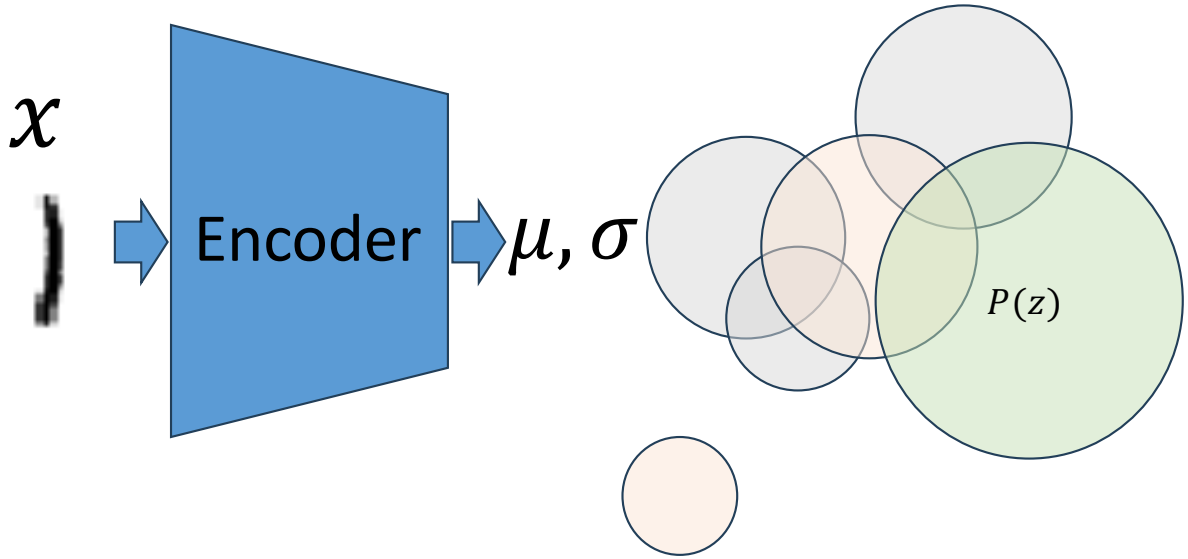


Improving the latent space….



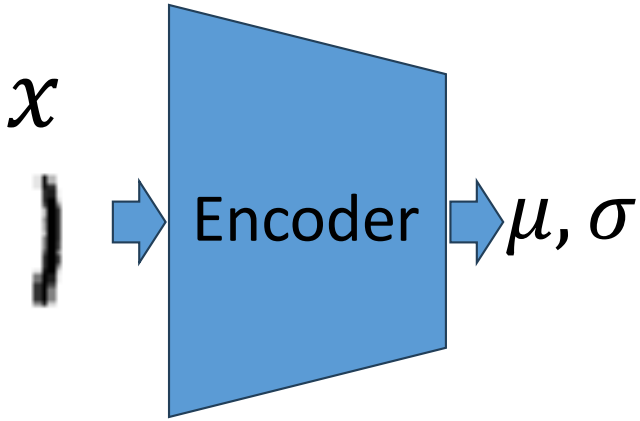
- We want contiguous regions for similar items
- We want smooth transitions

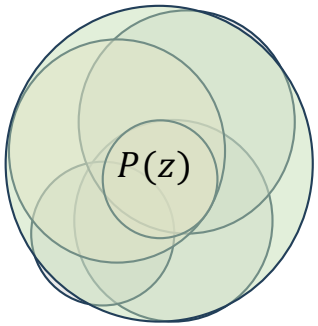
Loss function 1: shove everyone together



- Assume an isotropic Gaussian prior P(z)
- KL divergence Loss = $\mathcal{D}[Q(z|x)||P(z)]$

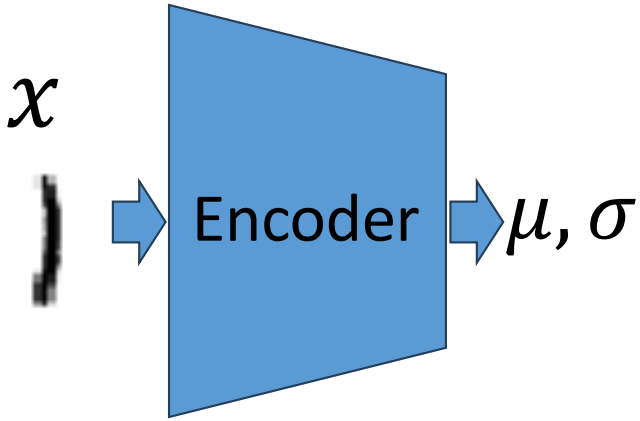
Loss function 1: shove everyone together

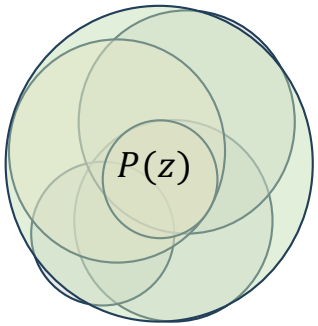




- Assume an isotropic Gaussian prior P(z)
- KL divergence Loss = $\mathcal{D}[Q(z|x)||P(z)]$

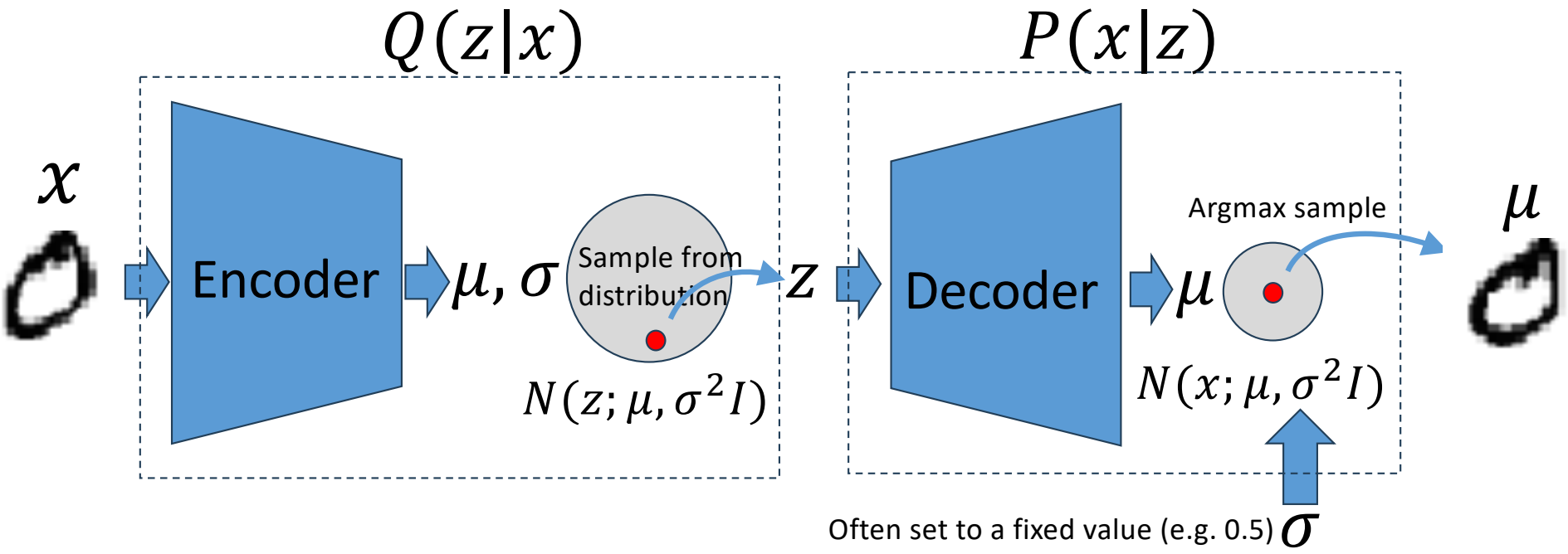
But wait…if they're all overlapped won't it be impossible to differentiate 0 from 1?





- Assume an isotropic Gaussian prior P(z)
- KL divergence Loss = $\mathcal{D}[Q(z|x)||P(z)]$

Loss function 2: match output to input



Mean Squared Error Loss: $\|x - \mu\|_2^2$

Losses in opposition

- The two loss functions have competing goals
- This led to Beta VAEs, where you scale the two losses to emphasize one over the other
- Often, you 1st minimize your MSE loss, and then you can start working on KL divergence loss.

Time for the math

Framing the problem as Bayesian inference

Let $\mathbf{x} = \mathrm{x}_1 \ldots \mathrm{x}_{\boldsymbol{n}}$ be a set of observed variables.

Let $\mathbf{z} = \mathrm{z}_1 \ldots \mathrm{z}_{\boldsymbol{m}}$ be a set of latent variables of interest.

We want to infer these latent variables from the evidence.

In other words: We want to know $p(\mathbf{z}|\mathbf{x})$.

$$p(\mathbf{z}|\mathbf{x}) = \frac{\boldsymbol{p}(\mathbf{z},\mathbf{x})}{\boldsymbol{p}(\mathbf{x})}$$

Set this up as an estimation problem.

- For the encoder, we want to approximate $\mathrm{P}(z|x)$
- Search through θ to find $Q^*(\mathbf{z})$, the density function optimizing this equation:

$$Q^*(z) = \underset{Q(\mathbf{z})\in\theta}{\operatorname{argmin}} D_{KL}(Q(\mathrm{z})||P(z|x))$$

Here, θ is the set of possible functions. Since this is going to be done using neural networks, we'll say it specifies the set of possible networks.

So how do we minimize this divergence?

Relating KL divergence to expected value

$$D_{KL}(Q(z)||P(z|x)) = \sum_z Q(z) log\left(\frac{Q(z)}{P(z|x)}\right)$$

$$= \sum_z Q(z)(\log Q(z) - \log P(z|x))$$

$$= E_{z\sim Q}[(\log Q(z) - \log P(z|x))]$$

Switching to Doersch's formulation

$$\mathcal{D}\left[Q(z)\|P(z|X)\right] = E_{z\sim Q}\left[\log Q(z) - \log P(z|X)\right]$$

KL Divergence is notated as D[A||B]

Q(z) is our user-defined distribution

P(z|X) is our unknown conditional distribution for z, given the evidence X.

Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

Doing some math…

Definition of KL divergence

$$\mathcal{D}[Q(z)||P(z|x)] = E_{z\sim Q}[(\log Q(z) - \log(P(z|x))]$$

Applying Bayes' rule

$$= E_{z\sim Q}\left[\log Q(z) - log\left(\frac{P(x|z)P(z)}{P(x)}\right)\right]$$

Using how division of logs works

$$= E_{z\sim Q}\left[\log Q(z) - log\big(P(x|z)\big) - \log\big(P(z)\big) + \log\big(P(x)\big)\right]$$

Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

Doing some math…

Where we left off

$$= E_{z\sim Q}\left[\log Q(z) - log\big(P(x|z)\big) - \log\big(P(z)\big) + \log\big(P(x)\big)\right]$$

Moving P(x) out of the expectation…since it doesn't depend on z

$$= E_{z\sim Q}\left[\log Q(z) - log\big(P(x|z)\big) - \log\big(P(z)\big)\right] + \log\big(P(x)\big)$$

Rearranging two of the terms

$$= E_{z\sim Q}\left[\log Q(z) - \log\big(P(z)\big) - log\big(P(x|z)\big)\right] + \log\big(P(x)\big)$$

Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

Doing some math…

Where we left off

$$= E_{z\sim Q}\left[\log Q(z) - \log\big(P(z)\big) - log\big(P(x|z)\big)\right] + \log\big(P(x)\big)$$

Remember how we defined KL divergence…

$$= \mathcal{D}[Q(z)||P(z)] + E_{z\sim Q}\left[-\log\big(P(x|z)\big)\right] + \log\big(P(x)\big)$$

Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)

Doing some math…

Bringing back the left side

$$\mathcal{D}[Q(z)||P(z|x)] = \mathcal{D}[Q(z)||P(z)] + E_{z\sim Q}\left[-\log\big(P(x|z)\big)\right] + \log\big(P(x)\big)$$

Negate both sides

$$-\mathcal{D}[Q(z)||P(z|x)] = -\mathcal{D}[Q(z)||P(z)] + E_{z\sim Q}\left[\log\big(P(x|z)\big)\right] - \log\big(P(x)\big)$$

Move terms

$$\log\big(P(x)\big) - \mathcal{D}[Q(z)||P(z|x)] = E_{z\sim Q}\big[\log\big(P(x|z)\big)\big] - \mathcal{D}[Q(z)||P(z)]$$

Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)

Conditioning latent generation on evidence

Note that X is fixed, and Q can be *any* distribution, not just a distribution which does a good job mapping X to the z's that can produce X. Since we're interested in inferring $P(X)$, it makes sense to construct a Q which *does* depend on X, and in particular, one which makes $\mathcal{D}\left[Q(z)\|P(z|X)\right]$ small:

I. E. Let's rewrite our formula to make Q(z) depend on x.

$$\log\big(P(x)\big) - \mathcal{D}[Q(z|x)||P(z|x)] = E_{z\sim Q(x|z)}\big[\log\big(P(x|z)\big)\big] - \mathcal{D}[Q(z|x)||P(z)]$$

Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

The Evidence Lower Bound

The approximation gap between the true posterior and the estimated one

The evidence lower bound (ELBO). This is what we can optimize.

$$\log\big(P(x)\big) - \mathcal{D}[Q(z|x)||P(z|x)] = E_{z\sim Q(x|z)}\big[\log\big(P(x|z)\big)\big] - \mathcal{D}[Q(z|x)||P(z)]$$

P(x) is fixed (it is the true probability distribution).

This ELBO is a lower bound on the value of P(x).

Raising the ELBO, lowers the approximation gap, until it hits 0.

Prior $P(x)$: We don't know this.

Prior $P(z)$: You choose…but it is typically set to a fixed standard isotropic Gaussian, $P(z) = \mathcal{N}(z; \mathbf{0}, \mathbf{I})$.

Encoder $Q_\phi(z|x)$: A neural network that takes input x and outputs the parameters of a conditional Gaussian distribution:

$$\mu, \sigma^2(x) = \text{Encoder}_\phi(x)$$

$$Q_\phi(z|x) = \mathcal{N}\left(z; \mu, \text{diag}(\sigma^2)\right)$$

Decoder $P_\theta(x|z)$: A neural network that maps latent vector z back to the data space.

Maximizing ELBO by minimizing divergence from P(z)

When both $Q_\phi(z|x)$ and $P(z)$ are multivariate Gaussians with diagonal covariance matrices, $\mathcal{D}_{KL}\left[Q_\phi(z|x) \,\|\, P(z)\right]$ can be computed **analytically:**

For latent dimensionality J:

$$\mathcal{D}_{KL}\left[Q_\phi(z|x) \,\|\, P(z)\right] = -\frac{1}{2}\sum_{j=1}^{J}\left(1 + \log\left(\sigma_j^2\right) - \mu_j^2 - \sigma_j^2\right)$$

This formula's gradients with respect to ϕ flow directly through standard backpropagation.

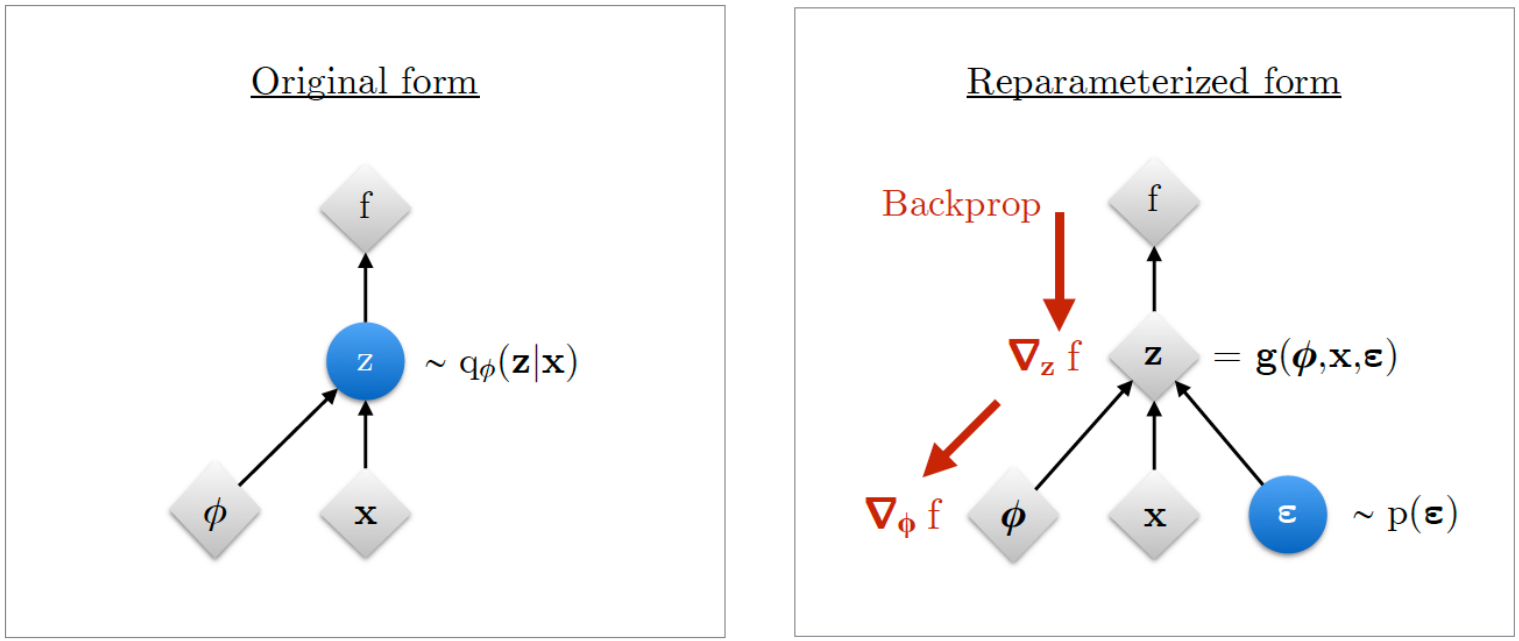
Estimating the Reconstruction Term

- Optimizing the reconstruction term $\mathbb{E}_{z\sim Q_\phi(z|x)}[\log P_\theta\,(x|z)]$ means backpropagating through the expectation over z.

- Sampling $z \sim Q_\phi(z|x)$ is non-differentiable, preventing gradient flow to the encoder parameters ϕ.

•The Solution (Reparameterization Trick)

- Make z a deterministic, differentiable function of the encoder outputs μ and σ^2, driven by an independent noise variable ϵ:
- $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- $z = g_\phi(x, \epsilon) = \mu + \sigma^2 \odot \epsilon$
- Now rewrite the expectation as:
- $\mathbb{E}_{z \sim Q_\phi(z|x)}[\log P_\theta(x|z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I})}\left[\log P_\theta\left(x|g_\phi(x,\epsilon)\right)\right]$
- Now, a Monte Carlo estimate with a single sample $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ is unbiased and fully differentiable with respect to both θ and ϕ.

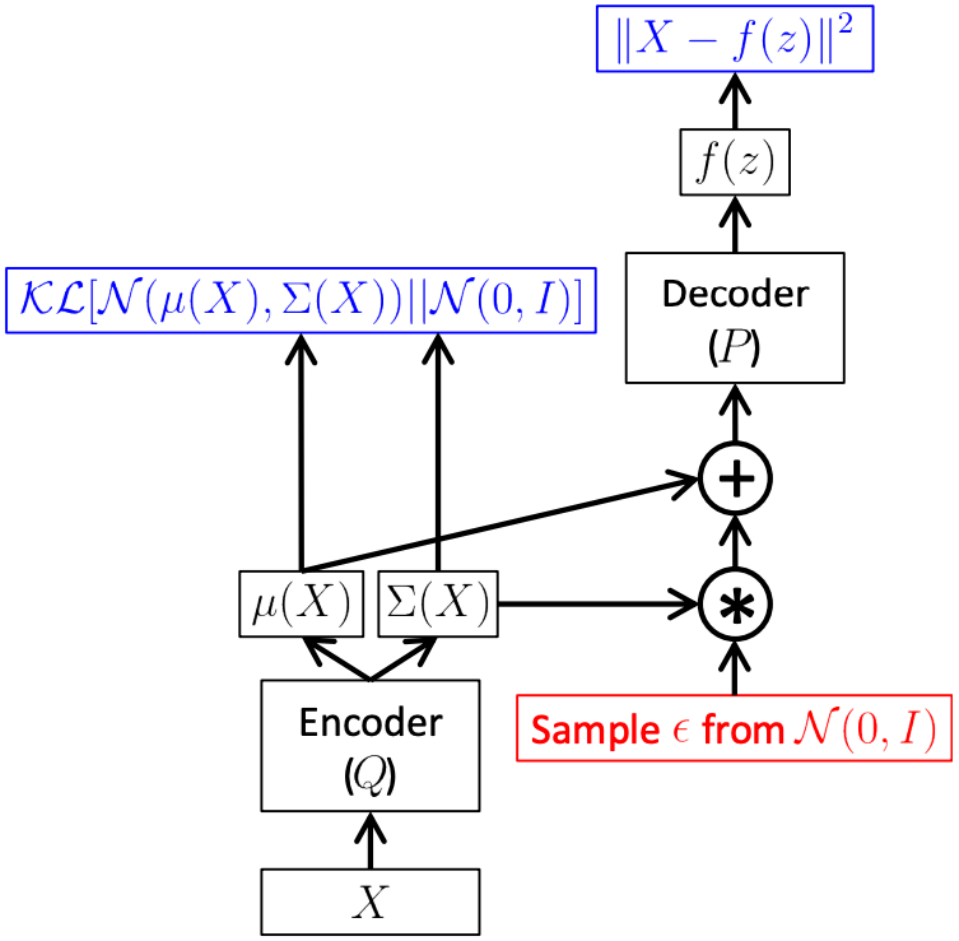


Kingma, D. P., & Welling, M. (2019). An introduction to variational autoencoders.

A graphical view

Blue box indicates loss measurement

Red box indicates non-differentiable



Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

Translating the decoder's output

- Wait…the math says that decoder is putting out probability distribution: $P_\theta(x|z)$.

- A probability distribution isn't a picture.

- So how do we get from that back to something we can call a picture (or a sound, or whatever)?

Evaluating the Decoder's output

1. Decoder Forward Pass: The decoder takes z as input and outputs the predicted mean vector μ (and optionally a variance σ^2) : $\mu = \text{Decoder}_\theta(z)$

2. The Decoder's Output Distribution: $P_\theta(x|z) = \mathcal{N}\left(x; \mu, \sigma^2\mathbf{I}\right)$ where $\mu, x \in \mathbb{R}^D$

3. Getting the Log-Likelihood: Evaluating the log density of a Gaussian gives:

$$\log P_\theta\,(x|z) = -\frac{D}{2}\log\left(2\pi\sigma^2\right) - \frac{1}{2\sigma^2}\|x-\mu\|_2^2$$

4. If we assume a fixed variance (e.g., $\sigma^2 = 0.5$), the log-likelihood covaries with the **Mean Squared Error (MSE)** loss between x and μ :

$$\log P_\theta\,(x|z) \propto -\|x-\mu\|_2^2$$

Unpacking how we got to $\log P_\theta\,(x|z) \propto -\|x-\mu\|_2^2$

- PDF of a Gaussian distribution for continuous data is:

$$P_\theta(x|z) = \frac{1}{(2\pi\sigma^2)^{D/2}} \exp\left(-\frac{\|x-\mu\|_2^2}{2\sigma^2}\right)$$

- If we take the natural log, then:

$$\log P_\theta\,(x|z) = -\frac{D}{2}\log\left(2\pi\sigma^2\right) - \frac{\|x-\mu\|_2^2}{2\sigma^2}$$

Dropping all the constants on the right hand side gives:

$$\log P_\theta\,(x|z) \propto -\|x-\mu\|_2^2$$

Flip the sign to make this minimize when things get better

$$\text{-}\log P_\theta\,(x|z) \propto \|x-\mu\|_2^2$$

Discussion points

- How do we make these things controllable?
- How do we navigate the latent space?
- What is “disentanglement”?

Adding class conditioning

Why do we want this?

- If I train a VAE on a big dataset of imaged (e.g. CFAR100), I want to be able to use it to generate an image of a particular class (like “dog”)
- Without class conditioning, how could I do this?
- Let’s think about the latent space….

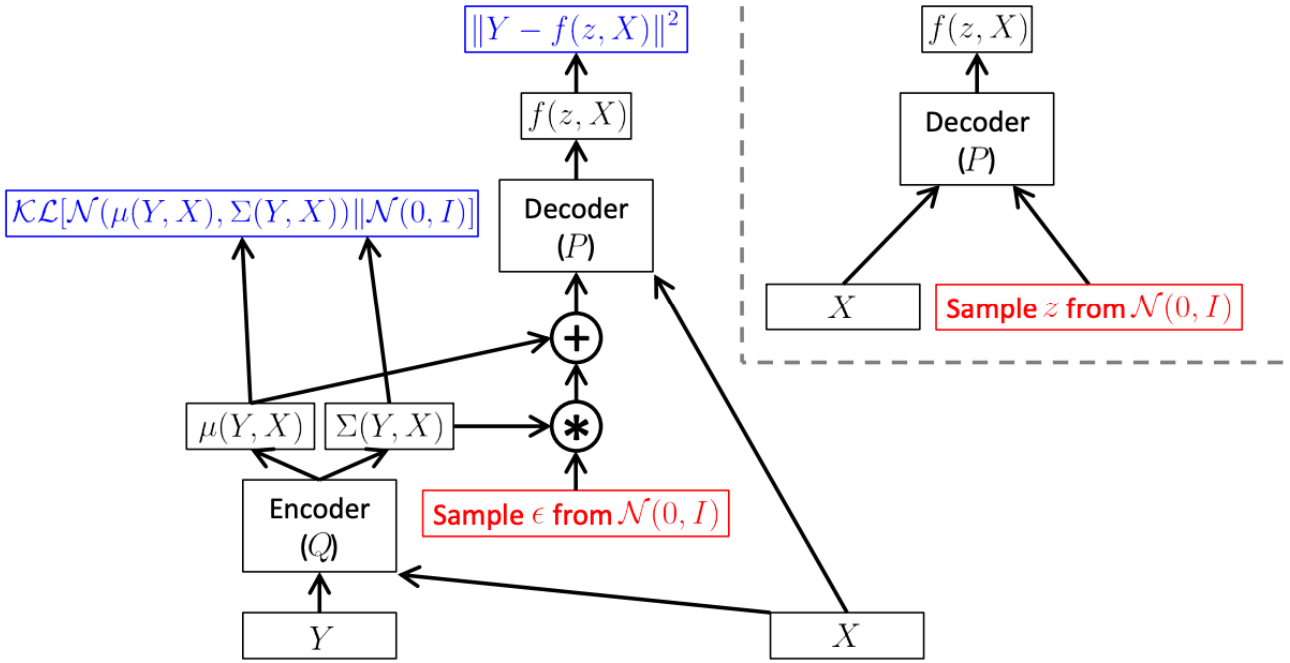
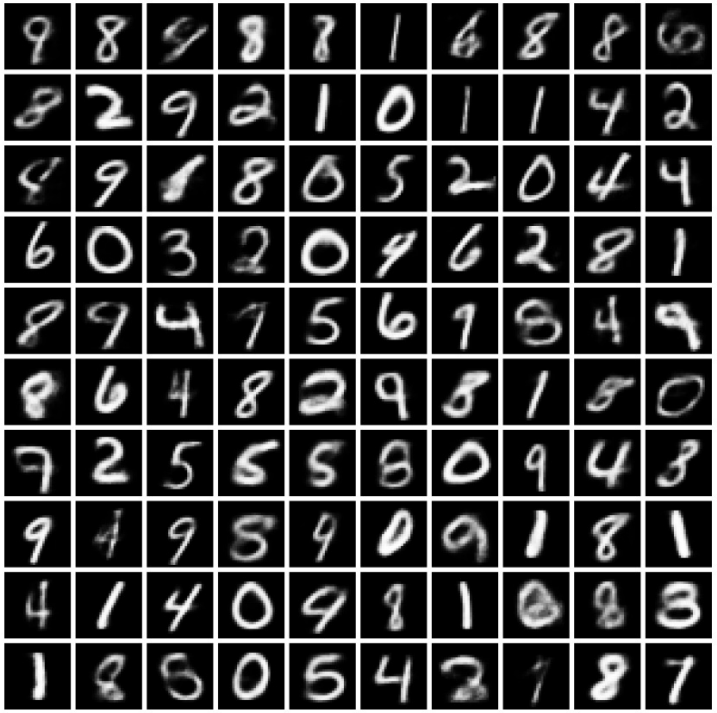


Figure 6: Left: a training-time conditional variational autoencoder implemented as a feedforward neural network, following the same notation as Figure 4. Right: the same model at test time, when we want to sample from $P(Y|X)$.

Doersch, Carl, "Tutorial on Variational Autoencoders " (2021)

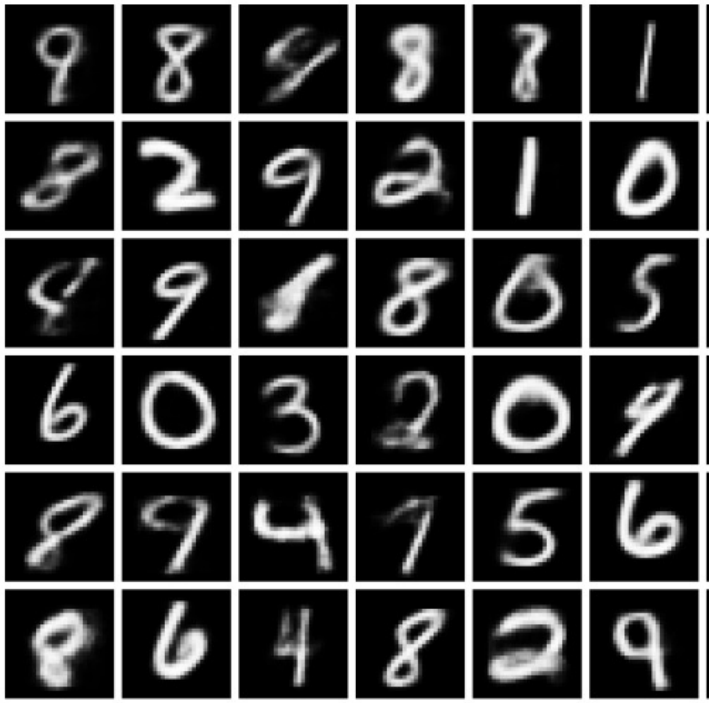
Example output

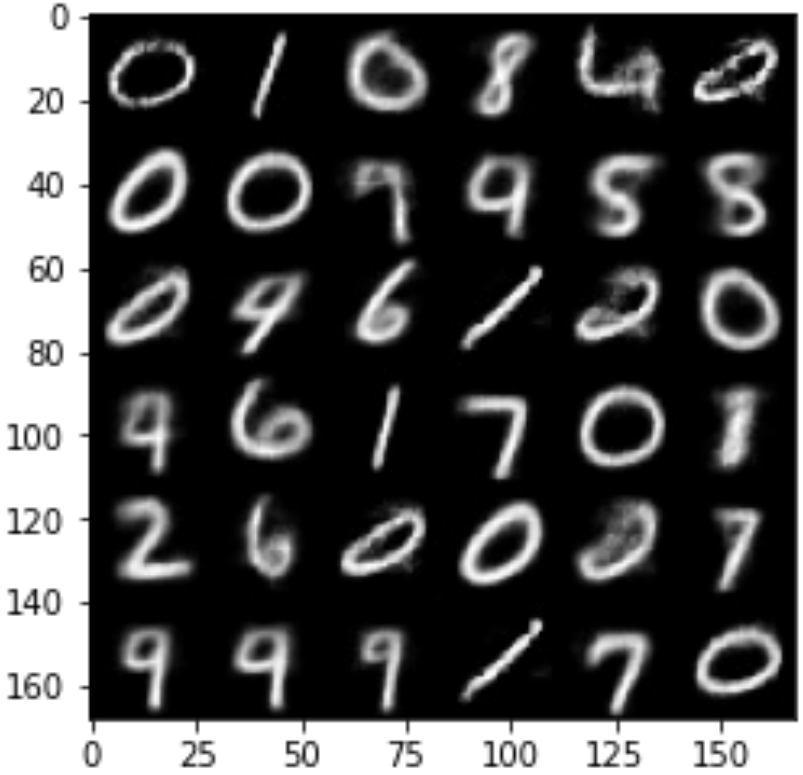
Unconditioned generation



Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)

VAE vs Autoencoder: unconditioned output

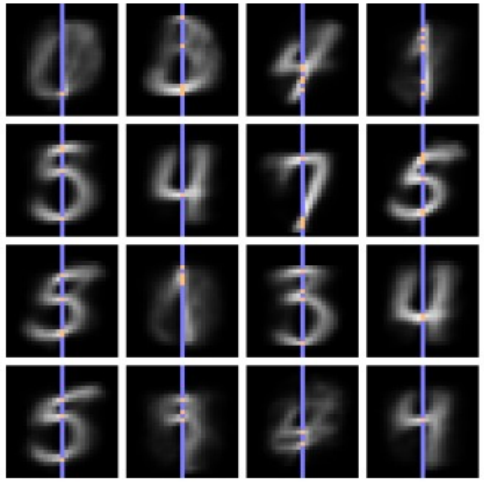




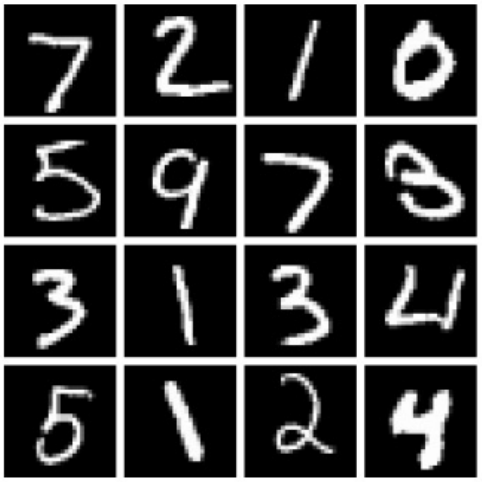
Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)

Conditional MNIST generation

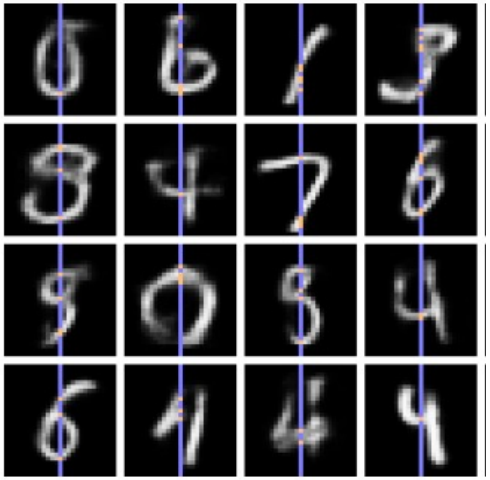
Regressor generation



Ground Truth



CVAE generation



Doersch, Carl, “Tutorial on Variational Autoencoders “ (2021)